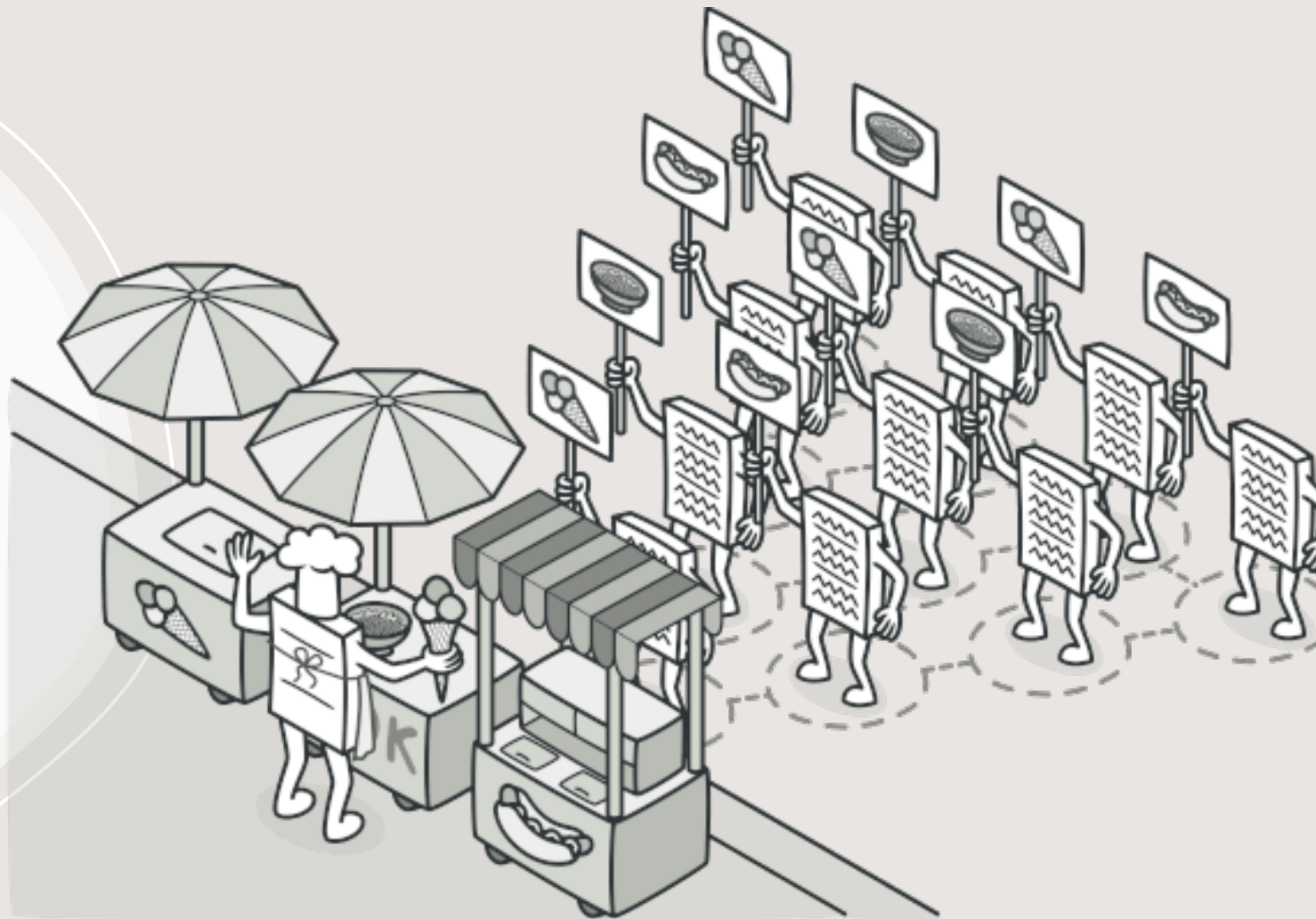


Visitor

Návštěvník



Hlavní idea

Visitor (návštěvník) je behaviorální návrhový vzor (vzor související s chováním), který vám umožňuje oddělit algoritmy od objektů, nad kterými pracují.



Návrhový vzor Visitor je s výhodou využívám v situacích, kdy je definována například hierarchie tříd se společným předkem. Tento předek pak deklaruje metodu, která je v každé podtřídě implementována odlišným způsobem. Zdrojový kód všech konkrétních implementací se z podtříd extrahuje a je přesunut do jedné nové třídy.



Problém

Představte si, že váš tým vyvíjí aplikaci, která pracuje s geografickými informacemi strukturovanými jako jeden komplexní graf. Každý uzel grafu může představovat ucelenou entitu, jako je město, ale také podrobnější věci, jako jsou průmyslová odvětví, památky atd.

Uzly jsou spojeny s ostatními, pokud mezi skutečnými objekty, které představují, existuje cesta. Každý typ uzlu je reprezentován svou vlastní třídou, přičemž každý konkrétní uzel je objekt.



Problém

V určitém okamžiku jste dostali úkol implementovat export grafu do formátu XML. Zpočátku se práce zdála docela jednoduchá. Plánovali jste přidat metodu exportu do každé třídy uzlů a poté využít rekurzi k procházení všech uzlů grafu a provedení metody exportu. Řešení se zdálo jednoduché a elegantní: díky polymorfismu jste kód, který volal metodu exportu, nepřipojovali ke konkrétním třídám uzlů.





Problém

Bohužel vám systémový architekt odmítl povolit změnu existujících tříd uzlů. Řekl, že kód je již v produkci a nechce riskovat jeho porušení kvůli potenciální chybě ve vašich změnách.

Kromě toho se zeptal, zda má smysl mít exportní kód XML ve třídách uzlů. Primární náplní těchto tříd byla práce s geodaty.

Přidání exportu XML tam tedy z tohoto úhlu pohledu úplně nepatří.



Problém

Pro odmítnutí byl ještě jeden důvod. Bylo vysoce pravděpodobné, že po implementaci této funkce vás někdo z marketingového oddělení požádá o možnost exportu do jiného formátu nebo požádá o nějaké jiné podivné věci. To by vás donutilo znovu změnit ty křehké třídy.



Řešení

Návrhový vzor **Visitor** navrhuje, umístit nové chování do samostatné třídy zvané *Visitor*, místo abychom se snažili je integrovat do existujících tříd. Původní objekt, který měl toto chování provést, je nyní předán jedné z metod návštěvníka jako argument, který metodě poskytuje přístup ke všem nezbytným datům obsaženým v objektu.



Řešení

Co když lze toto chování provést nad objekty různých tříd? Například v našem případě s XML exportem bude skutečná implementace pravděpodobně trochu odlišná napříč různými třídami uzlů. Třída návštěvníka tedy může definovat ne jednu, ale sadu metod, z nichž každá může mít argumenty různých typů, jako je tento:

```
class ExportVisitor implements Visitor is
    method doForCity(City c) { ... }
    method doForIndustry(Industry f) { ... }
    method doForSightSeeing(SightSeeing ss) { ... }
    // ...
```



Řešení

Ale jak přesně bychom tyto metody nazvali, zvláště když se zabýváme celým grafem? Tyto metody mají různé signatury, takže nemůžeme použít polymorfismus. Abychom vybrali správnou metodu návštěvníka, která je schopna zpracovat daný objekt, museli bychom zkontrolovat jeho třídu. Není to však špatný přístup?

```
foreach (Node node in graph)
    if (node instanceof City)
        exportVisitor.doForCity((City) node)
    if (node instanceof Industry)
        exportVisitor.doForIndustry((Industry) node)
    // ...
}
```



Řešení

Možná se nabízí otázka, proč nepoužíváme přetěžování metod? Tedy dát všem metodám stejný název, i když podporují různé sady parametrů. Bohužel, ani za předpokladu, že to náš programovací jazyk podporuje (jako Java a C#), nám to nepomůže. Vzhledem k tomu, že přesná třída objektu uzlu je předem neznámá, mechanismus přetížení nebude schopen určit správnou metodu k provedení. Výchozí bude metoda, která přebírá objekt základní třídy **Node**.



Řešení

Návrhový vzor Visitor však tento problém řeší. Využívá techniku zvanou **Double Dispatch**, která pomáhá zavolat správnou metodu na objektu. Místo toho, abychom nechali klienta vybrat správnou verzi metody, nabízí se otázka, zda není vhodné delegovat tuto volbu na objekty, které předáváme návštěvníkovi jako argument. Cesta tedy je „akceptovat“ návštěvníka a říci mu, jaký způsob návštěvy by měl být proveden.



Řešení

```
// Client code
foreach (Node node in graph)
    node.accept(exportVisitor)

// City
class City is
    method accept(Visitor v) is
        v.doForCity(this)
    // ...

// Industry
class Industry is
    method accept(Visitor v) is
        v.doForIndustry(this)
    // ...
```



Řešení

Je třeba ale přiznat , že nakonec jsme museli změnit třídy uzlů. Ale alespoň je změna triviální a umožňuje nám přidat další chování, aniž bychom znovu změnili kód.

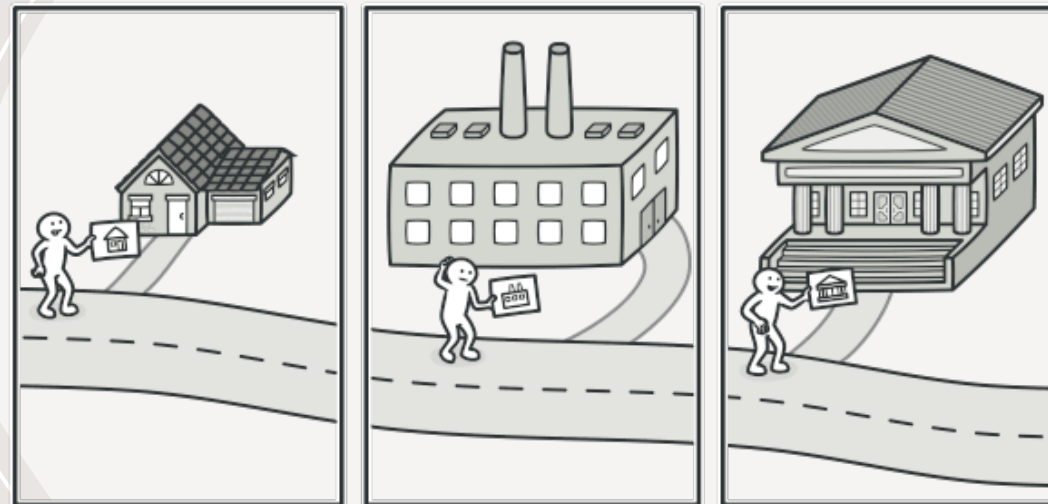
Pokud nyní extrahujeme společné rozhraní pro všechny návštěvníky, všechny existující uzly mohou pracovat s jakýmkoli návštěvníkem, kterého do aplikace představíte. Pokud zjistíte, že zavádíte nové chování související s uzly, vše, co musíte udělat, je implementovat novou třídu návštěvníků.



Analogie v reálném světě

Představte si zkušeného pojišťovacího agenta, který touží získat nové zákazníky. Může navštívit každou budovu ve čtvrti a pokusit se prodat pojištění každému, koho potká. V závislosti na typu organizace, která v budově sídlí, může nabídnout specializované pojištění:

- Pokud se jedná o obytnou budovu, prodává zdravotní pojištění.
- Pokud je to banka, prodává pojištění proti krádeži.
- Pokud je to kavárna, prodává pojištění proti požáru a povodni.





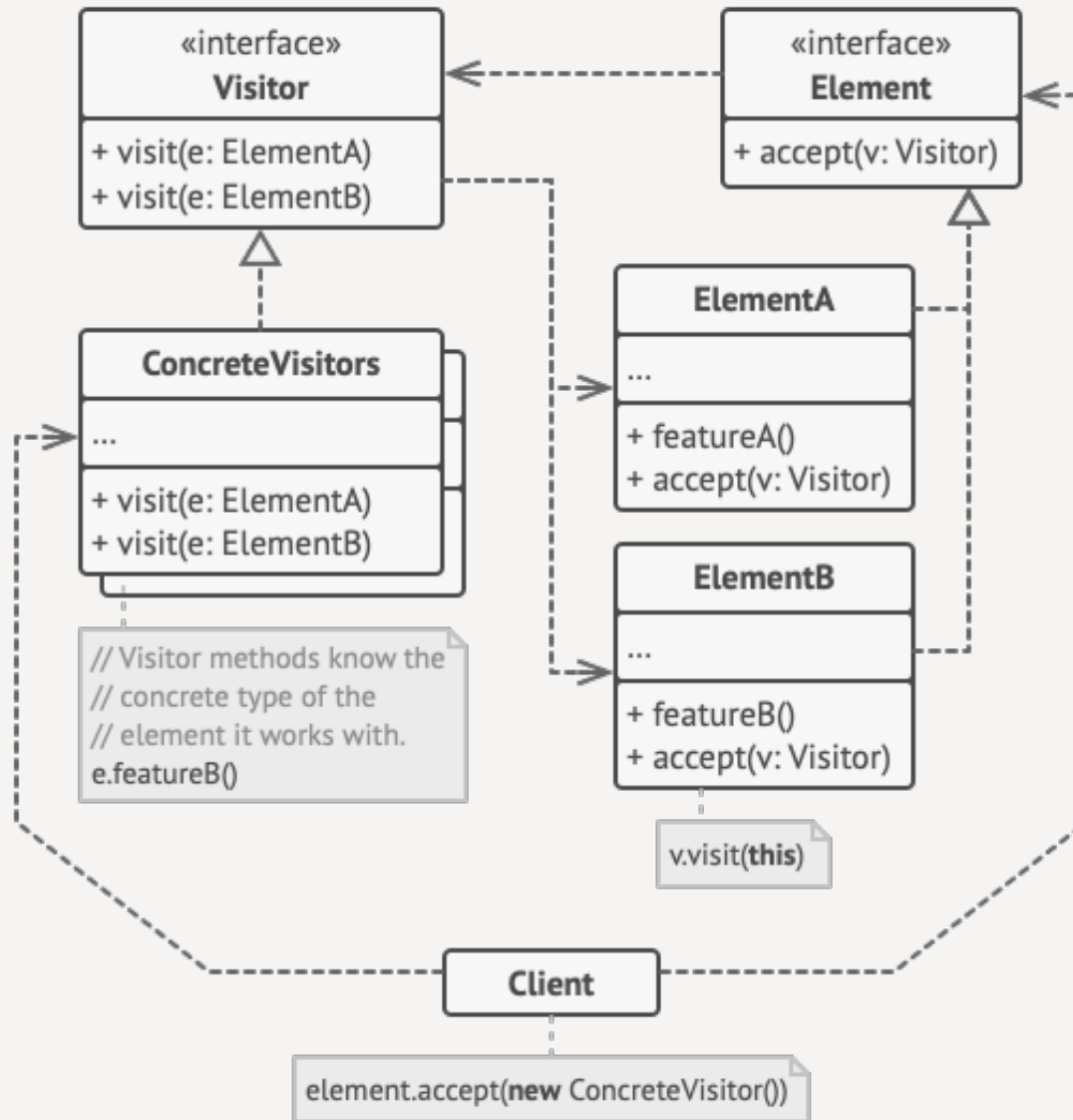
Výhody a nevýhody



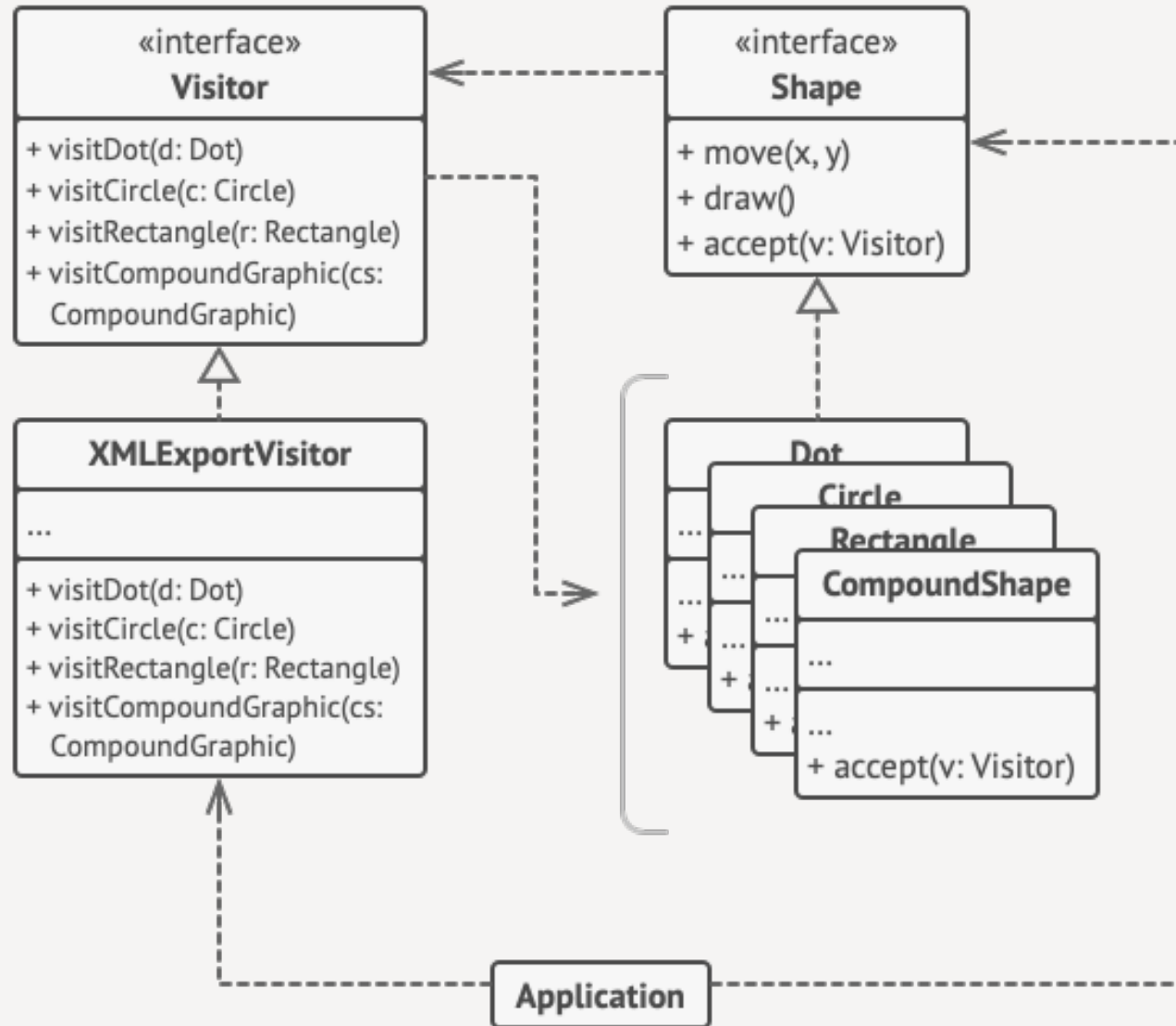
- Princip otevřený/uzavřený. Můžete zavést nové chování, které může pracovat s objekty různých tříd, aniž byste tyto třídy měnili.
- Princip jednotné odpovědnosti. Do stejné třídy můžete přesunout více verzí stejného chování.



- Pokaždé, když se třída přidá do hierarchie prvků nebo z ní odebere, musíte aktualizovat všechny návštěvníky.
- Návštěvníci mohou postrádat nezbytný přístup k soukromým polím a metodám prvků, se kterými mají pracovat.



Struktura



Implementace



```
import upa.fei.dp.behavioral.visitor.visit.Visitor;

public interface Shape {
    void move(int x, int y);
    void draw();
    String accept(Visitor visitor);
}
```

```
public class Circle extends Dot {
    private int radius;

    public Circle(int id, int x, int y, int radius) {
        super(id, x, y);
        this.radius = radius;
    }

    @Override
    public String accept(Visitor visitor) { return visitor.visitCircle(this); }

    public int getRadius() { return radius; }
}
```

```
public class Dot implements Shape {
    private int id;
    private int x;
    private int y;

    public Dot() {}

    public Dot(int id, int x, int y) {
        this.id = id;
        this.x = x;
        this.y = y;
    }

    @Override
    public void move(int x, int y) {}

    @Override
    public void draw() {}

    @Override
    public String accept(Visitor visitor) { return visitor.visitDot(this); }

    public int getX() { return x; }

    public int getY() { return y; }

    public int getId() { return id; }
}
```

```
public class Rectangle implements Shape {
    private int id;
    private int x;
    private int y;
    private int width;
    private int height;

    public Rectangle(int id, int x, int y, int width, int height) {
        this.id = id;
        this.x = x;
        this.y = y;
        this.width = width;
        this.height = height;
    }

    @Override
    public String accept(Visitor visitor) { return visitor.visitRectangle(this); }

    @Override
    public void move(int x, int y) {}

    @Override
    public void draw() {}

    public int getId() { return id; }

    public int getX() { return x; }

    public int getY() { return y; }

    public int getWidth() { return width; }

    public int getHeight() { return height; }
}
```



```
import java.util.ArrayList;
import java.util.List;

public class CompoundShape implements Shape {
    public int id;
    public List<Shape> children = new ArrayList<>();

    public CompoundShape(int id) { this.id = id; }

    @Override
    public void move(int x, int y) {}

    @Override
    public void draw() {}

    public int getId() { return id; }

    @Override
    public String accept(Visitor visitor) { return visitor.visitCompoundGraphic( cg: this); }

    public void add(Shape shape) { children.add(shape); }
}
```



```
public interface Visitor {  
    String visitDot(Dot dot);  
  
    String visitCircle(Circle circle);  
  
    String visitRectangle(Rectangle rectangle);  
  
    String visitCompoundGraphic(CompoundShape cg);  
}
```

```
public class XMLExportVisitor implements Visitor {  
    public String export(Shape... args) {  
        StringBuilder sb = new StringBuilder();  
        sb.append("<?xml version='1.0' encoding='utf-8'?'>" + "\n");  
        for (Shape shape : args) {  
            sb.append(shape.accept( visitor: this)).append("\n");  
        }  
        return sb.toString();  
    }  
  
    public String visitDot(Dot d) {  
        return "<dot>" + "\n" +  
            "    <id>" + d.getId() + "</id>" + "\n" +  
            "    <x>" + d.getX() + "</x>" + "\n" +  
            "    <y>" + d.getY() + "</y>" + "\n" +  
            "</dot>";  
    }  
  
    public String visitCircle(Circle c) {  
        return "<circle>" + "\n" +  
            "    <id>" + c.getId() + "</id>" + "\n" +  
            "    <x>" + c.getX() + "</x>" + "\n" +  
            "    <y>" + c.getY() + "</y>" + "\n" +  
            "    <radius>" + c.getRadius() + "</radius>" + "\n" +  
            "</circle>";  
    }  
  
    public String visitRectangle(Rectangle r) {  
        return "<rectangle>" + "\n" +  
            "    <id>" + r.getId() + "</id>" + "\n" +  
            "    <x>" + r.getX() + "</x>" + "\n" +  
            "    <y>" + r.getY() + "</y>" + "\n" +  
            "    <width>" + r.getWidth() + "</width>" + "\n" +  
            "    <height>" + r.getHeight() + "</height>" + "\n" +  
            "</rectangle>";  
    }  
  
    public String visitCompoundGraphic(CompoundShape cg) {  
        return "<compound_graphic>" + "\n" +  
            "    <id>" + cg.getId() + "</id>" + "\n" +  
            "    _visitCompoundGraphic(cg) +  
            "</compound_graphic>";  
    }  
  
    private String _visitCompoundGraphic(CompoundShape cg) {  
        StringBuilder sb = new StringBuilder();  
        for (Shape shape : cg.children) {  
            String obj = shape.accept( visitor: this);  
            // Proper indentation for sub-objects.  
            obj = "    " + obj.replace( target: "\n", replacement: "\n    ") + "\n";  
            sb.append(obj);  
        }  
        return sb.toString();  
    }  
}
```



```
public class ClientVisitor {  
  
    public static void main(String[] args) {  
        Dot dot = new Dot( id: 1, x: 10, y: 55);  
        Circle circle = new Circle( id: 2, x: 23, y: 15, radius: 10);  
        Rectangle rectangle = new Rectangle( id: 3, x: 10, y: 17, width: 20, height: 30);  
  
        CompoundShape compoundShape = new CompoundShape( id: 4);  
        compoundShape.add(dot);  
        compoundShape.add(circle);  
        compoundShape.add(rectangle);  
  
        CompoundShape c = new CompoundShape( id: 5);  
        c.add(dot);  
        compoundShape.add(c);  
  
        export(circle, compoundShape);  
    }  
  
    private static void export(Shape... shapes) {  
        XMLExportVisitor exportVisitor = new XMLExportVisitor();  
        System.out.println(exportVisitor.export(shapes));  
    }  
}
```



Zdroje

<https://refactoring.guru/design-patterns>

<http://voho.eu/wiki/navrhovy-vzor/>

<https://www.oodesign.com/>